



PROGRAMAÇÃO APLICADA EM .NET
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
RELATÓRIO DE AULAS PRÁTICAS

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem _____

PROGRAMAÇÃO APLICADA EM .NET

Orientações para relatórios e critérios de avaliação

Base tecnológica: .NET 10 LTS, Visual Studio 2026 18.0, C# 14, ASP.NET Core 10 e EF Core 10

Ambiente de desenvolvimento

Visual Studio 2026 versão 18.0 ou superior, com a carga de trabalho ASP.NET e desenvolvimento Web. Utilizar SDK .NET 10 atualizado e C# 14. Instalar ou disponibilizar as ferramentas de diagnóstico utilizadas nas atividades, como dotnet-counters e dotnet-trace, além dos pacotes e ferramentas necessários ao BenchmarkDotNet, ao EF Core e ao provedor de banco de dados adotado. O Visual Studio Profiler pode ser utilizado nas atividades de diagnóstico e comparação de desempenho. Registrar no relatório as versões efetivamente utilizadas do Visual Studio, SDK, pacotes, banco de dados e demais ferramentas que influenciem a reprodução dos resultados.

Organização do conjunto

Este documento define o que o aluno deve explicar, documentar, testar e entregar em cada unidade. Cada rubrica soma 10,0 pontos: fundamentação (2,0), estrutura do código (3,0), funcionamento (3,0) e aprimoramentos (2,0).

Escala de pontuação: em cada critério, atribuir 100%, 75%, 50%, 25% ou 0% do peso para atendimento integral, predominante, parcial, mínimo ou inexistente. A mesma falha deve ser descontada somente no critério mais diretamente afetado. As extensões são opcionais para concluir a atividade, mas necessárias para obter a pontuação integral de Aprimoramentos.

As oito atividades usam o domínio didático Hub de Processamento de Pedidos. O professor pode substituir o domínio, desde que preserve os contratos, testes e critérios.

Atividade	Eixo
1	Pattern matching, atributos, reflexão controlada, records e DTOs imutáveis
2	Span<T>, Memory<T>, pooling e caminhos críticos limitados por E/S e medidos
3	Consultas previsíveis, projeções, tracking, consultas compiladas e JSON
4	Channels, SemaphoreSlim, coleções concorrentes e backpressure
5	BenchmarkDotNet, counters, traces, p95/p99 e testes de carga
6	Modularização, IoC, fronteiras e versionamento compatível de APIs
7	SignalR, reconexão, streaming, cache de resposta e ETags
8	WebApplicationFactory, bancos efêmeros, analisadores e revisão automatizada

Verificação: As atividades são cumulativas. Antes de iniciar uma unidade, mantenha uma versão executável da unidade anterior.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: Pattern matching, atributos, reflexão controlada, records e DTOs imutáveis	Relatório 1
--	--	--------------------

Unidade 1 – Linguagem avançada em C# e .NET

Objetivo: Modelar os contratos e o domínio do Hub de Pedidos com tipos imutáveis, classificação por padrões e descoberta controlada de processadores.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Padrões de tipo, propriedade, relacionais, lógicos, posicionais e de lista; ordem dos braços e exaustividade de expressões switch.
- Atributos como metadados declarativos e reflexão como mecanismo de inspeção em tempo de execução.
- Riscos de varreduras amplas por reflexão: custo de inicialização, tipos inesperados, carregamento de assemblies e tratamento de exceções.
- Diferenças entre record class, record struct e classes comuns, incluindo igualdade por valor e cópia com with.
- Imutabilidade superficial versus profunda e cuidado com coleções mutáveis dentro de records.

- Separação entre entidade, value object, DTO de entrada e DTO de saída; conversões explícitas nas fronteiras.

3. Código-fonte e evidências

- Records, value objects, DTOs e mapeadores produzidos para pedidos e itens.
- Expressão switch que classifique pedidos e tabela de casos cobertos.
- Atributo customizado e catálogo de processadores construído com reflexão restrita.
- Testes de igualdade, cópia, validação e descoberta de processadores.

4. Testes e validações obrigatórias

- Classificação de pedido normal, prioritário, inválido e caso-limite.
- Expressão de pattern matching sem braço inalcançável e com comportamento explícito para entrada desconhecida.
- Record copiado com with sem alterar o original.
- DTO inválido rejeitado antes de criar o objeto de domínio.
- Reflexão limitada ao assembly/namespace previsto e execução sem duplicar processadores.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de linguagem	2,0	Explica padrões, records, imutabilidade, atributos e limites da reflexão.
Estrutura e contratos	3,0	Tipos, DTOs, mapeadores e catálogo preservam fronteiras e invariantes.
Funcionamento e evidências	3,0	Classificação, descoberta, validação e testes operam nos casos definidos.

Critério	Peso	Descrição
Aprimoramentos	2,0	Inclui generator, teste arquitetural ou melhoria equivalente bem justificada.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: Span<T>, Memory<T>, pooling e caminhos críticos limitados por E/S e medidos	Relatório 2
--	---	--------------------

Unidade 2 – Performance e alocação eficiente

Objetivo: Reduzir cópias e alocações no recebimento de lotes de pedidos, comprovando ganhos e custos com benchmarks reproduzíveis.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Span<T> e ReadOnlySpan<T> como visões de memória com restrições de tempo de vida e uso assíncrono.
- Memory<T> e ReadOnlyMemory<T> em operações assíncronas; propriedade, consumo e duração do empréstimo do buffer.
- Uso de ArrayPool<T>/MemoryPool<T>, devolução garantida, limpeza quando há dados sensíveis e riscos de reter memória alugada.
- Alocação, cópia, pressão de GC e diferença entre melhorar microcódigo e remover I/O ou roundtrips dominantes.
- Hot paths I/O-bound: leitura incremental, buffers, cancelamento, flushes e limites de tamanho.

- Benchmark como experimento: baseline, aquecimento, isolamento, diagnósticos de memória e interpretação sem generalização indevida.

3. Código-fonte e evidências

- Implementações baseline e otimizada do parser ou normalizador de lote.
- BenchmarkDotNet com tempo, alocações, configuração, versão e ambiente registrados.
- Código de aluguel/devolução de buffer e demonstração do caminho de erro/cancelamento.
- Tabela de decisão que aceite ou rejeite a otimização com base em evidência e complexidade.

4. Testes e validações obrigatórias

- Equivalência funcional entre baseline e versão otimizada.
- Entrada vazia, incompleta, acima do limite e com UTF-8 multibyte.
- Buffer devolvido ao pool mesmo quando ocorre exceção ou cancelamento.
- Benchmark executado em Release fora do depurador, com baseline explícita.
- Teste de fluxo grande sem carregar o corpo completo desnecessariamente em memória.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de memória	2,0	Explica tempo de vida, propriedade dos buffers, pooling, GC e limites de otimização.
Estrutura e segurança	3,0	Buffers, streams, cancelamento e devolução são tratados sem corrupção ou vazamento.

Critério	Peso	Descrição
Funcionamento e medições	3,0	Baseline, equivalência e benchmarks reproduzíveis sustentam a conclusão.
Aprimoramentos	2,0	Inclui comparação adicional ou melhoria equivalente baseada em dados.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: Consultas previsíveis, projeções, tracking, consultas compiladas e JSON	Relatório 3
--	---	--------------------

Unidade 3 – LINQ/EF Core avançado e mapeamentos

Objetivo: Persistir e consultar pedidos com EF Core, analisando SQL, roundtrips, tracking e impacto dos mapeamentos adotados.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Tradução de LINQ para SQL e diferença entre expressão executada no banco e processamento no cliente.
- Projeções enxutas, paginação, agregações, prevenção de N+1 e limites de carregamento relacionado.
- Tracking, AsNoTracking e resolução de identidade conforme intenção de leitura ou atualização.
- Cache interno de consultas e uso criterioso de EF.CompileQuery/EF.CompileAsyncQuery em consultas realmente quentes.

- Leitura do SQL gerado, parâmetros, quantidade de comandos e correlação com índices e plano do banco.
- Tipos complexos e JSON como decisão dependente de versão/provedor; impacto em consulta, atualização, indexação e portabilidade.

3. Código-fonte e evidências

- Modelo, configurações e migration de pedidos, itens e metadados.
- Consultas de painel com projeção, paginação e AsNoTracking, acompanhadas do SQL gerado.
- Comparação entre consulta normal e compilada para um caso repetitivo.
- Análise do mapeamento relacional versus JSON e suas limitações no provedor escolhido.

4. Testes e validações obrigatórias

- Consulta paginada retorna somente colunas e linhas necessárias.
- Quando o resultado é atualizado pelo mesmo DbContext, a consulta usa tracking; outra estratégia exige anexação explícita documentada e testada.
- Nenhuma consulta N+1 no relatório de pedidos e itens.
- Parâmetros do usuário não alteram a forma da expressão dinamicamente com constantes indevidas.
- Mapeamento complexo/JSON testado apenas quando suportado pelo provedor e versão registrados.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de EF Core	2,0	Explica tradução, tracking, projeção, compilação e mapeamentos.
Estrutura das consultas	3,0	LINQ, DTOs, parâmetros, paginação e modelo são claros e previsíveis.
Funcionamento e evidências	3,0	SQL, testes e medições demonstram eficiência e ausência de N+1.
Aprimoramentos	2,0	Inclui comparação de paginação, comandos ou melhoria equivalente bem analisada.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: Channels, SemaphoreSlim, coleções concorrentes e backpressure	Relatório 4
--	---	--------------------

Unidade 4 – Concorrência e pipelines

Objetivo: Processar pedidos por uma fila limitada, controlando produtores, consumidores, acesso ao banco e encerramento da aplicação.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Modelo produtor/consumidor e diferenças entre fila concorrente simples e Channel<T> assíncrono.
- Canal limitado, modos de preenchimento e backpressure como proteção de memória e estabilidade.
- SemaphoreSlim para limitar concorrência lógica; diferença entre paralelismo permitido e capacidade real do banco.
- Coleções concorrentes, operações atômicas e risco de composições check-then-act não atômicas.
- Cancelamento, conclusão do writer, drenagem, tratamento de falhas e shutdown gracioso.

- Relação entre tamanho da fila, consumidores, pool de conexões, latência, throughput e rejeição controlada.

3. Código-fonte e evidências

- Configuração do canal limitado e política escolhida quando a fila está cheia.
- Worker com consumidores, cancelamento, tratamento de falhas e conclusão coordenada.
- Limite de concorrência para persistência e justificativa em relação ao pool de conexões.
- Gráficos ou tabela de fila, throughput, descartes, rejeições e latência sob duas configurações.

4. Testes e validações obrigatórias

- No modo Wait, o produtor aguarda; nos modos de descarte, a política é observada; quando houver rejeição explícita, TryWrite e a resposta HTTP são testados.
- Quantidade de operações no banco nunca excede o limite definido.
- Falha de um item não encerra silenciosamente todos os consumidores.
- Cancelamento impede novos itens e permite estratégia explícita para os pendentes.
- Nenhum pedido processado duas vezes no cenário de teste sem política de repetição.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de concorrência	2,0	Explica canais, backpressure, limites, atonicidade e encerramento.
Estrutura e confiabilidade	3,0	Produtores, consumidores, escopos e sincronização evitam vazamentos e corridas.

Critério	Peso	Descrição
Funcionamento e evidências	3,0	Capacidade, teto de concorrência, falhas e shutdown funcionam nos testes.
Aprimoramentos	2,0	Inclui prioridade, idempotência, lote ou melhoria equivalente mensurada.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: BenchmarkDotNet, counters, traces, p95/p99 e testes de carga	Relatório 5
--	--	--------------------

Unidade 5 – Diagnóstico e perfilamento

Objetivo: Diagnosticar o Hub sob carga, estabelecer metas de latência e investigar gargalos com evidências reproduzíveis.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Diferença entre microbenchmark, profiling, monitoramento de processo e teste de carga da aplicação.
- BenchmarkDotNet: isolamento, warmup, iterações, baseline, diagnósticos e variabilidade.
- dotnet-counters como triagem de CPU, GC, alocação, exceções, ThreadPool e métricas customizadas.
- dotnet-trace/EventPipe para coleta temporal e identificação de métodos, esperas e starvation.
- Percentis p50, p95 e p99, cauda de latência e cuidado com médias que escondem outliers.

- Teste de carga orientado a hipótese: modelo de chegada, concorrência, duração, aquecimento, massa de dados e critério de parada.

3. Código-fonte e evidências

- Plano de medição com hipótese, métricas, carga, ambiente e critérios de sucesso.
- Relatório de BenchmarkDotNet do método selecionado e captura de counters durante o teste de carga.
- Trace analisado com evidência do gargalo ou ausência dele.
- Tabela p50/p95/p99, throughput, erros e conclusão antes/depois de uma intervenção.

4. Testes e validações obrigatórias

- Ferramentas executadas contra build Release no ambiente registrado.
- Carga contém aquecimento e janela de medição separados.
- p95/p99 calculados sobre amostra identificada, não inferidos apenas pela média.
- Cenário registra erros e timeouts, não apenas respostas bem-sucedidas.
- Uma intervenção é mantida somente se melhora a meta sem regressão funcional relevante.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de medição	2,0	Distingue ferramentas, percentis, carga, hipótese e limitações.
Desenho experimental	3,0	Ambiente, scripts, amostra, metas e comparação são reproduzíveis.

Critério	Peso	Descrição
Diagnóstico e evidências	3,0	Counters, trace e percentis sustentam a conclusão e preservam correção.
Aprimoramentos	2,0	Inclui saturação, profiler, orçamento ou melhoria equivalente coerente.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: Modularização, IoC, fronteiras e versionamento compatível de APIs	Relatório 6
--	---	--------------------

Unidade 6 – Arquitetura aplicada

Objetivo: Separar responsabilidades do Hub e evoluir o contrato HTTP sem romper um cliente que depende da versão anterior.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Módulo como fronteira de responsabilidade e diferença entre organização por pasta e dependência realmente controlada.
- Inversão de controle, composition root, tempos de vida e risco de service locator ou dependência cativa.
- Boundaries de domínio, contratos internos, eventos e prevenção de vazamento de detalhes de persistência.
- Mudanças compatíveis e incompatíveis em payloads, status, semântica, ordenação e comportamento.
- Estratégias de versionamento por URL, cabeçalho ou query e política de versão padrão/depreciação.

- Compatibilidade para front-ends: contrato publicado, coexistência, telemetria de uso e remoção planejada.

3. Código-fonte e evidências

- Diagrama simples de módulos e direção das dependências.
- Composition root e registros de DI com tempos de vida justificados.
- Versões v1 e v2 do endpoint selecionado e matriz de mudanças.
- Teste de compatibilidade provando que o cliente v1 continua funcional.

4. Testes e validações obrigatórias

- Camada de domínio compila sem referência à API ou EF Core.
- Nenhum serviço scoped é capturado por singleton.
- v1 e v2 coexistem e produzem contratos documentados.
- Cliente v1 continua interpretando a resposta sem mudança silenciosa de semântica.
- Versão ausente ou não suportada segue política explícita e testada.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação arquitetural	2,0	Explica módulos, IoC, boundaries, compatibilidade e versionamento.
Estrutura e contratos	3,0	Dependências, tempos de vida e versões estão separados e documentados.
Funcionamento e evidências	3,0	Grafo, v1/v2 e cliente legado funcionam nos testes definidos.

Critério	Peso	Descrição
Aprimoramentos	2,0	Inclui teste arquitetural, adaptador ou melhoria equivalente integrada.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: SignalR, reconexão, streaming, cache de resposta e ETags	Relatório 7
--	--	--------------------

Unidade 7 – Tempo real e UX

Objetivo: Notificar o cliente sobre o processamento de pedidos e reduzir transferências de leitura com cache e validação condicional corretos.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- SignalR como abstração de comunicação em tempo real e diferença entre hub, conexão, grupo e método de cliente.
- Notificações, streaming, ordenação, duplicidade e necessidade de reconciliação com uma fonte HTTP autoritativa.
- Estados de conexão, reconexão automática, backoff, revalidação da autorização, restauração de grupos e feedback ao usuário.
- Cache de resposta versus cache de dados e riscos de armazenar respostas personalizadas ou autorizadas.
- ETag como validador de representação; If-None-Match, 304 e If-Match/412 para pré-condições quando adotadas.

- Invalidação, variação por parâmetros/headers e coerência entre cache, versão da API e atualização em tempo real.

3. Código-fonte e evidências

- Hub, grupos e evento tipado de mudança de status.
- Cliente com estados conectado, reconectando e desconectado e estratégia de reconciliação.
- Resposta com ETag e exemplos de GET normal, If-None-Match e 304.
- Política de output caching, chaves de variação, invalidação e rotas excluídas.

4. Testes e validações obrigatórias

- Cliente recebe atualização somente após autorização validada no servidor; a associação ao grupo não substitui a autorização.
- Reconexão revalida a autorização, restaura a assinatura permitida e reconcilia eventos possivelmente perdidos.
- ETag muda quando a representação muda e permanece estável quando não muda.
- If-None-Match válido retorna 304 sem corpo; valor obsoleto retorna representação atual.
- Output cache é registrado após autenticação e autorização e não compartilha respostas privadas entre usuários.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de tempo real/HTTP	2,0	Explica hubs, reconexão, cache, ETags e pré-condições.

Critério	Peso	Descrição
Estrutura e segurança	3,0	Eventos, grupos, contratos e cache preservam autorização e coerência.
Funcionamento e evidências	3,0	Notificação, reconciliação, 304 e invalidação funcionam nos testes.
Aprimoramentos	2,0	Inclui streaming, If-Match ou melhoria equivalente bem testada.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Aplicada em .NET Título da Aula: WebApplicationFactory, bancos efêmeros, analisadores e revisão automatizada	Relatório 8
--	---	--------------------

Unidade 8 – Qualidade contínua

Objetivo: Criar uma rede de segurança executável para contratos, persistência, concorrência e estilo, integrada ao fluxo de revisão.

1. Orientações gerais de entrega

- Produzir relatório individual ou em equipe, conforme definição do professor, com 2 a 4 páginas de conteúdo, sem contar capa, código integral e referências.
- Usar linguagem própria, relacionar conceitos ao código produzido e identificar claramente as decisões técnicas.
- Entregar o repositório ou pacote solicitado, o relatório em formato editável e, quando exigido, também em PDF, além das evidências de execução.
- Não inserir senhas, cookies, tokens, connection strings sensíveis, dados pessoais ou capturas que revelem credenciais.
- Indicar versões de SDK, pacotes, banco e ferramentas que influenciem a reprodução do resultado.

2. Resumo teórico

Explicar, com palavras próprias e vinculando a teoria à atividade:

- Testes de integração como execução da aplicação real com substituições explícitas de infraestrutura.
- WebApplicationFactory<TEntryPoint>, cliente de teste, configuração por ambiente e troca controlada de serviços.
- Banco efêmero: isolamento, criação e remoção, fidelidade do provedor e diferenças entre SQLite em memória e o banco produtivo.
- Pirâmide/portfólio de testes: unidade, integração, contrato, carga e poucos testes ponta a ponta.
- Analisadores Roslyn, regras CA/IDE, .editorconfig, severidade e diferença entre estilo, qualidade e segurança.

- Revisão automatizada como verificação reproduzível: restore, build, dotnet format -verify-no-changes, testes, artefatos e diagnóstico de falha.

3. Código-fonte e evidências

- Fábrica customizada, banco efêmero e fixtures de integração.
- Testes HTTP de criação, consulta, versão, ETag e fluxo assíncrono selecionado.
- .editorconfig e configuração de analisadores com justificativa das severidades.
- Workflow ou script de revisão que falhe em teste/análise e produza resultados legíveis.

4. Testes e validações obrigatórias

- Cada teste inicia com estado conhecido e não depende da ordem de execução.
- Banco efêmero preserva conexão/vida útil necessária e é descartado ao final.
- Contrato v1 e resposta condicional continuam protegidos por integração.
- Uma violação selecionada de analisador falha o build conforme a política.
- O pipeline falha e impede a publicação quando o build, os testes ou a análise falham.

5. Referências

- Listar somente as fontes efetivamente consultadas, priorizando documentação oficial, normas e especificações.
- Apresentar as referências em padrão ABNT e informar data de acesso para documentação online.

Critérios de avaliação

Critério	Peso	Descrição
Fundamentação de qualidade	2,0	Explica integração, fidelidade do banco, analisadores e gates.
Estrutura e isolamento	3,0	Fábrica, fixtures, dados e regras são determinísticos e legíveis.

Critério	Peso	Descrição
Funcionamento e evidências	3,0	Testes, analisadores e revisão automatizada falham ou passam como definido.
Aprimoramentos	2,0	Inclui contrato, paralelismo, artefatos ou melhoria equivalente responsável.

Verificação: Nota final: soma dos quatro critérios, conforme a escala e as regras gerais definidas na abertura deste documento.